

Asignatura: Programación II

Trabajo Practico Nº 1

Comisión: 2603	**Ciclo lectivo**: 2026

Docentes: Miguel Silva

Gestión Fast Food

Integrantes

Apellido y Nombre

Corvalán Portillo, Carla Sofía

García, Pablo Gustavo

Ciancio Vicente Reynaldo

Fecha de entrega: 13 de Julio de 2026

DATOS DEL PROYECTO

- **Repositorio de GitHub:** https://github.com/CorvalanPortilloINSPT/java-solid-poo-project

Informe Técnico de Arquitectura y Diseño del Sistema: fastFood. Tema 13

Documento de Especificación Técnica
Contexto de Aplicación: Arquitectura en Capas y Persistencia en Archivos (.txt)
Tecnología: Java SE (Programación Orientada a Objetos)

1. Diseño del Sistema

El sistema fastFood está desarrollado bajo una arquitectura modular estructurada en capas independientes. Esta separación de responsabilidades garantiza la mantenibilidad del código, facilita las pruebas unitarias y desacopla por completo la interfaz de usuario de los mecanismos de almacenamiento. Las capas que componen el ecosistema del software se detallan a continuación:

- **Modelos (Entities / Domain Objects):** Representan las entidades de negocio esenciales de la aplicación (por ejemplo, Usuario, Producto, Pedido, Cliente). Contienen los atributos privados, constructores y métodos de acceso (getters/setters), encapsulando el estado puro de los datos sin lógica de infraestructura.
- **Controladores (Controllers):** Actúan como intermediarios y coordinadores principales del flujo de datos entre las Vistas y la capa de Servicios. Capturan las peticiones del usuario, delegan el procesamiento correspondiente y determinan la siguiente acción o pantalla a renderizar.
- **DAO (Data Access Object) & BD (Base de Datos simulada):** La capa DAO es la única responsable de realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) sobre el sistema de almacenamiento. Interactúa directamente con la carpeta o archivos de texto (.txt) que actúan como la base de datos simulada de la aplicación, abstrayendo las lecturas y escrituras físicas.
- **DTO (Data Transfer Object):** Estructuras de datos optimizadas para el transporte de información entre capas, evitando exponer directamente las entidades del modelo de dominio hacia la interfaz de usuario, mitigando riesgos de acoplamiento innecesario.
- **Servicios (Services):** Contienen toda la lógica de negocio, validaciones complejas, reglas operativas del restaurante y orquestan las transacciones llamando a uno o múltiples DAOs.
- **Vistas (Views):** Representan la interfaz de usuario de la aplicación. Capturan las entradas del operador mediante la consola de comandos utilizando la clase Scanner y muestran los menús interactivos, tablas de datos y confirmaciones.

- **Excepciones (Exceptions): (A desarrollar)** Clases especializadas que heredan de Exception o RuntimeException para mapear fallos específicos del negocio (ej. DniDuplicadoException, IngredienteDuplicadoException), garantizando un control de errores centralizado y robusto.

2. Decisiones de Implementación

Para la construcción de este sistema se tomaron decisiones técnicas fundamentadas en los requisitos del entorno académico y operativo, optimizando el rendimiento sin recurrir a frameworks externos complejos:

- **Persistencia basada en archivos de texto planos (.txt):** Se implementó una base de datos embebida simulada mediante archivos planos estructurados (por ejemplo, registros delimitados por comas o punto y coma). Esta decisión elimina la dependencia de un motor SGBD externo (como MySQL o PostgreSQL), facilitando la portabilidad absoluta del proyecto. La capa DAO se encarga de parsear líneas de texto hacia objetos Java y viceversa de manera transparente.
- **Manejo de entrada de datos seguro y control del Buffer (Scanner):** Dado que la interfaz es por consola, se estandarizó el uso de la clase Scanner. Para mitigar el error común de salto de líneas al alternar entre la lectura de tipos primitivos numéricos (nextInt(), nextDouble()) y cadenas (nextLine()), se decidió limpiar el buffer de entrada de forma explícita mediante un Scanner.nextLine() residual inmediatamente después de capturar un valor numérico, o realizar lecturas completas de línea mediante nextLine() y posterior parseo manual (Integer.parseInt()), garantizando la estabilidad de la UI.

3. Uso de los Principios de Programación Orientada a Objetos (POO)

El proyecto se diseñó como un ecosistema puramente orientado a objetos, aplicando los pilares fundamentales del paradigma:

- **Abstracción:** Se identificaron los actores y elementos del dominio del restaurante de comida rápida, abstrayendo sus características esenciales en clases abstractas y concretas. Las entidades del mundo real se modelaron omitiendo complejidades irrelevantes, enfocándose únicamente en las propiedades computacionales críticas para el negocio.
- **Encapsulamiento:** Todas las variables de miembro (atributos) de las clases de modelo y servicios se definieron con el modificador de acceso private. El acceso y la modificación del estado interno de los objetos se realiza exclusivamente a través de métodos públicos de interfaz (getters y setters), aplicando validaciones dentro de los mutadores para prevenir estados inconsistentes en memoria.
- **Herencia:** Se estructuró una jerarquía de clases eficiente para evitar la duplicación de código (DRY - Don't Repeat Yourself). Por ejemplo, en el control de usuarios, clases como Cliente y Empleado extienden de una superclase común Persona , heredando atributos compartidos como ID, nombre, correo y contraseñas, permitiendo centralizar la lógica común.
- **Polimorfismo:** Aplicado a través de la implementación de interfaces y la sobrescritura de métodos (@Override). La capa de persistencia define contratos abstractos que pueden ser resueltos de múltiples maneras (por ejemplo, una interfaz de persistencia que puede ser implementada por un

DAO que lee archivos de texto o, en el futuro, por un DAO que apunte a una base de datos relacional real sin alterar el resto de la aplicación).

4. Aplicación de los Principios S.O.L.I.D.

Para garantizar un diseño de software limpio, escalable y tolerante al cambio, se incorporaron los principios S.O.L.I.D. en la estructura arquitectónica:

- **S - Single Responsibility Principle (Principio de Responsabilidad Única):** Cada clase del sistema fastFood tiene una única razón para cambiar. Los modelos solo contienen datos, los DAOs solo manejan operaciones de persistencia en disco, los servicios procesan exclusivamente lógica de negocio y las vistas controlan únicamente la interacción de la consola. Ninguna clase mezcla responsabilidades cruzadas.
- **O - Open/Closed Principle (Principio de Abierto/Cerrado):** El sistema está abierto a la extensión pero cerrado a la modificación. Gracias al uso intensivo de interfaces en la capa de servicios y DAO, si se requiere añadir un nuevo método de pago o un nuevo tipo de almacenamiento, se crean nuevas clases que implementan estas interfaces sin necesidad de alterar, recompilar o romper el código existente.
- **L - Liskov Substitution Principle (Principio de Sustitución de Liskov):** Cualquier subclase o implementación (por ejemplo, Cliente extendiendo de Persona) puede sustituir por completo a su superclase sin alterar la corrección del programa. Los métodos heredados respetan rigurosamente el comportamiento esperado por la clase base.
- **I - Interface Segregation Principle (Principio de Segregación de Interfaces):** Se prefieren interfaces pequeñas, específicas y enfocadas a tareas puntuales en lugar de interfaces masivas o genéricas. Esto evita que las clases implementadoras se vean obligadas a codificar o dejar en blanco métodos que no necesitan.
- **D - Dependency Inversion Principle (Principio de Inversión de Dependencias):** Los módulos de alto nivel (como los Controladores y Servicios) no dependen directamente de los módulos de bajo nivel (como las implementaciones concretas de los archivos de texto de los DAOs). Ambos dependen de abstracciones (interfaces), reduciendo drásticamente el acoplamiento y facilitando el intercambio de componentes.

5. Aclaraciones Relevantes y Buenas Prácticas

Como anotaciones finales sobre el desarrollo técnico del software fastFood, se destacan los siguientes criterios de calidad:

- **Manejo Centralizado de Excepciones de Negocio:** La separación de excepciones controladas evita el uso de bloques try-catch redundantes y desorganizados en la UI. Las fallas operativas se propagan elegantemente hacia el controlador o la vista, donde son presentadas de forma amigable al usuario final.
- **Convenciones de Nombres Estandarizadas:** Se respetan estrictamente las convenciones de código de Java: camelCase para nombres de variables y métodos, PascalCase para nombres de clases e

interfaces, y UPPERCASE_WITH_UNDERSCORES para constantes inmutables, asegurando una lectura homogénea.

- **Persistencia Atómica:** Para evitar la corrupción de datos en los archivos .txt durante operaciones concurrentes o cancelaciones abruptas, las escrituras en disco se realizan de manera síncrona asegurando que el flujo de datos (BufferedWriter/FileWriter) se cierre correctamente (mediante estructuras try-with-resources) al finalizar cada operación.